Course
Game Design: Theory and Practice

•••

5th Year Computer Science Engineering - Software Engineering

Instructor: Dr. Delloul

Semester 9 / 2026-2027

Chapter 2
Sprites and Tiles

Chapter Plan

- Introduction
- Creating and Visualising Sprites
- Creating and Visualizing Tiles
- Tilemap Scrolling
- Visualizing a Scene
- Isometric Tiles
- Example of a 2D Game

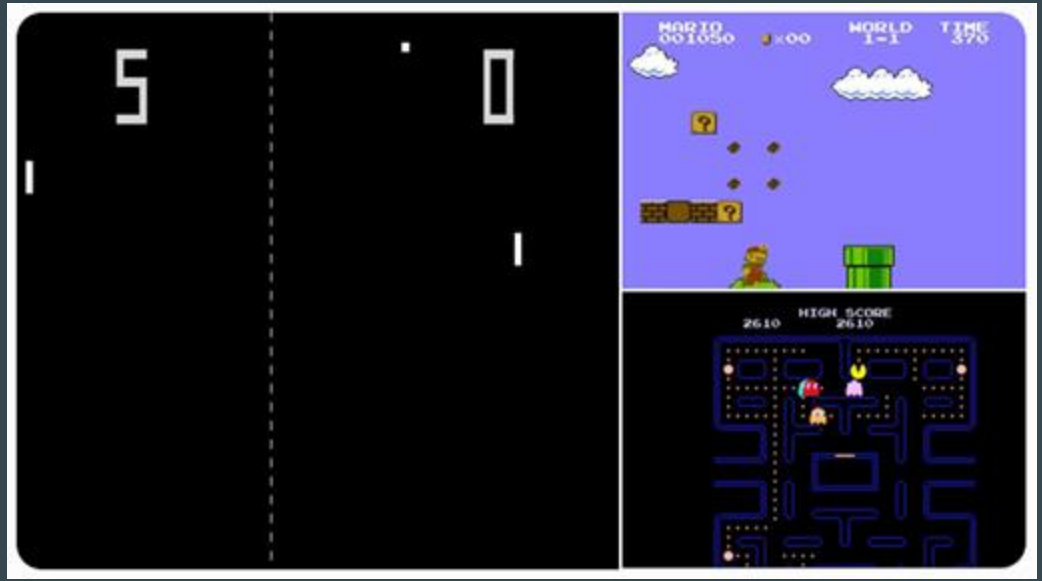
Introduction

Video games have evolved from simple, flat graphics into realistic, immersive 3D worlds. This evolution happened through advances in computer technology, graphics, and game design.

Introduction

The first video games used simple 2D graphics, because computer hardware had very limited memory and processing power.

Everything was represented on a flat screen using pixels. Developers created characters, objects, and environments using small graphical elements.



Introduction

As hardware improved, developers could display more colors, larger sprites, and smoother animations.

Games introduced more detailed backgrounds, scrolling environments, and increasingly sophisticated visual effects.







Introduction

With the arrival of more powerful consoles and computers, developers began exploring new ways to create depth hence the 3D games.



Introduction

Although video game graphics have evolved from simple pixels to complex 3D environments, the fundamental challenge remains the same: how do we represent, organize, and display visual elements efficiently?

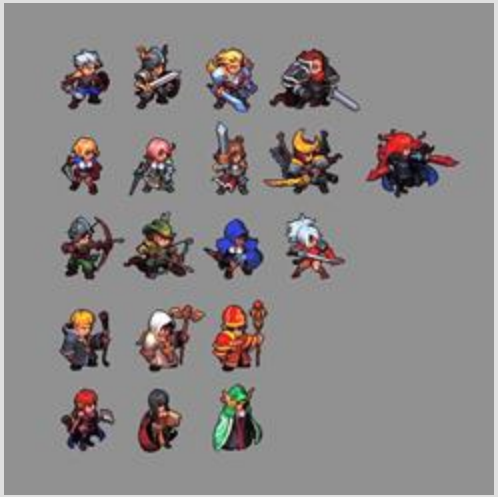
Sprites and tiles are two of the fundamental techniques that helped make video games possible. Understanding them allows us to learn how 2D game graphics work, how environments are constructed, and how game engines manage visual resources.

Creating and Visualizing Sprites

A sprite is an image representing a visual object, often a character or an object that moves independently.



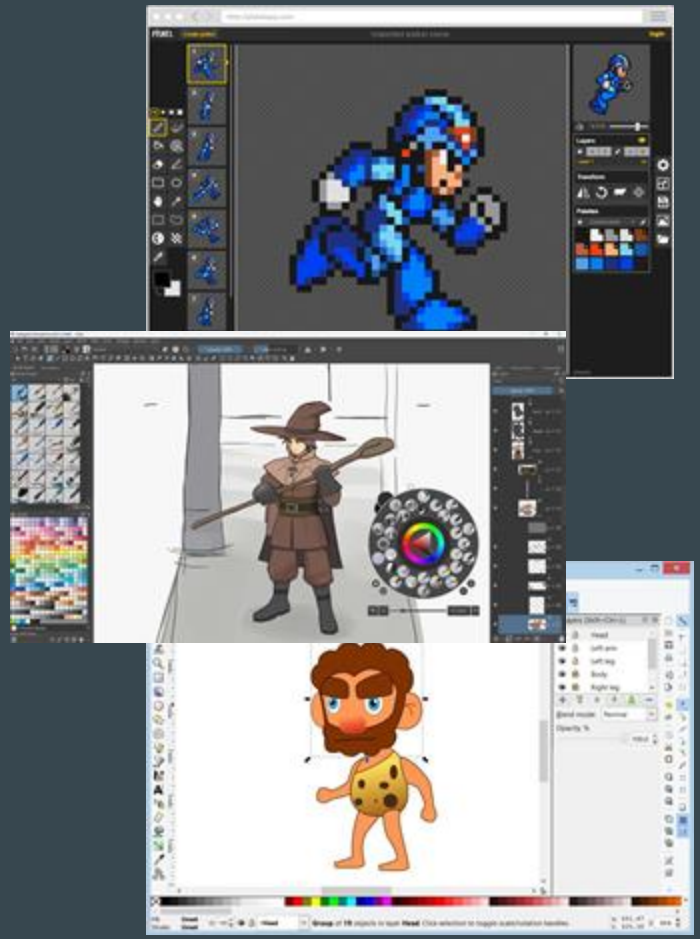




Creating and Visualizing Sprites

There are several ways to create a sprite, depending on the desired visual style and the tools available:

1. Pixel-art editors: Designed for pixel-by-pixel drawing, color palettes, and frame-by-frame animation. (Aseprite, Piskel)
2. Digital drawing software: Useful for hand-drawn sprites, painted characters, and more detailed illustrations. (GIMP, Krita..)
3. Vector graphics software: Useful for creating shapes and illustrations that can be resized without losing quality. (Inkscape)



Creating and Visualizing Sprites

To create a sprite we must follow the process:

1. **Choose the dimension of the sprite:** generally 32x32 or 64x64 pixels.
2. **Draw the silhouette:** Make the character recognizable, even before adding details.
3. **Add colors and details:** Use colors to distinguish the body, clothes, face, and other features.
4. **Keep the background transparent:** This lets the game display the character without a rectangular background.
5. **Export the sprite:** PNG is a common format because it supports transparency.

Creating and Visualizing Sprites

To animate the sprite, we need to draw every step that represents the movement on images of same size. Each image is called a ‘frame’.

Displaying these frames rapidly in sequence creates the illusion of movement.

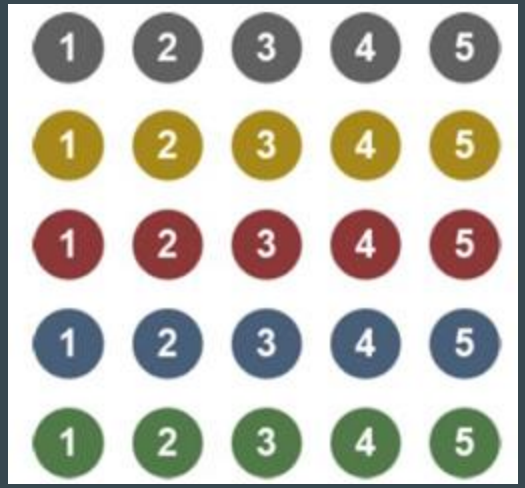
A collection of frames arranged in one image is called a **sprite sheet**. During gameplay, the appropriate frame sequence is selected to be animated at the appropriate moment.

Example: When the player clicks the Walk button, the row of the sprite sheet containing the appropriate frames is selected, and the frames are displayed one by one to create the walking animation.

Creating and Visualizing Sprites

Examples of sprite sheets:







Creating and Visualizing Sprites

Exercise:

Write the algorithm that animates “running” from a sprite sheet of size 320x96 where the sprite is of size 32x32, where the running sequence is on the second row.

Creating and Visualizing Sprites

```
frame_size = 32;
frames = 320 / frame_size; // in this case 10
img = load("spritesheet.png");
for (i = 0; i < frames; i++) {
    display(img, i * frame_size, 1 * frame_size,
            frame_size, frame_size);
        wait(100);
}
```

Creating and Visualizing Tiles

A tile is a small image used to build a larger game environment by placing multiple tiles next to each other.



Creating and Visualizing Tiles

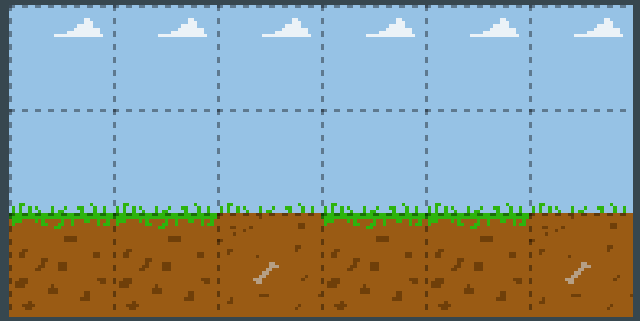
To create a 2D game environment:

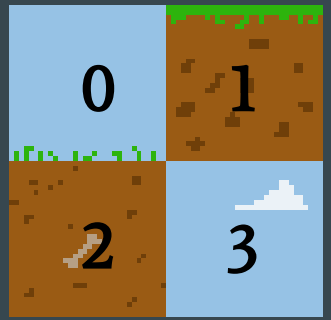
1. Create individual tiles: grass, water, stone, wood, dirt...etc;
2. Collect them into a **tileset;**
3. Place the tiles on a grid to create the game world, called a **tilemap;**
4. The game engine displays the map as a complete environment.



Creating and Visualizing Tiles

This means that we need to have a matrix of the positions of each tile in the game world: tilemap.






```
3,3,3,3,3,3
0,0,0,0,0,0
1,1,2,1,1,2
```





Tiles vs. Sprites

Tiles	Sprites
Usually represents part of the environment.	Usually represents a character, object, or animation.
Often repeated across a grid.	Often placed independently.
Examples: grass, floor, wall.	Examples: player, enemy, coin.

Tilemap Scrolling

Tilemap Scrolling

In a 2D game, a tilemap can be larger than the screen. Instead of displaying the entire map at once, the game shows only a portion of it. As the player moves, the visible portion changes. **This is called tilemap scrolling.**

There are three main elements:

- Tilemap: the complete game world, built from tiles.
- Camera / viewport: the area visible to the player.
- Scrolling: moving the viewport across the tilemap to reveal new areas.

Tilemap Scrolling

The camera:

- Follows an object (like the character in platform games);
- Is controlled by the player (like strategy games);
- Finds which portion of the game map to be displayed.

Tilemap Scrolling



Tilemap Scrolling



Tilemap Scrolling

1. Horizontal scrolling

The camera moves left or right as the player progresses. Common in platform games.



2. Multi-directional scrolling

The camera moves up, down, left, or right. Common in top-down adventure games



Tilemap Scrolling

The principle is simple:

1. Create a tilemap larger than the screen.
2. Choose the camera's position in the world.
3. Draw the tiles visible inside the camera's viewport.
4. Update the camera position when the player moves.
5. Display the newly visible portion of the tilemap.

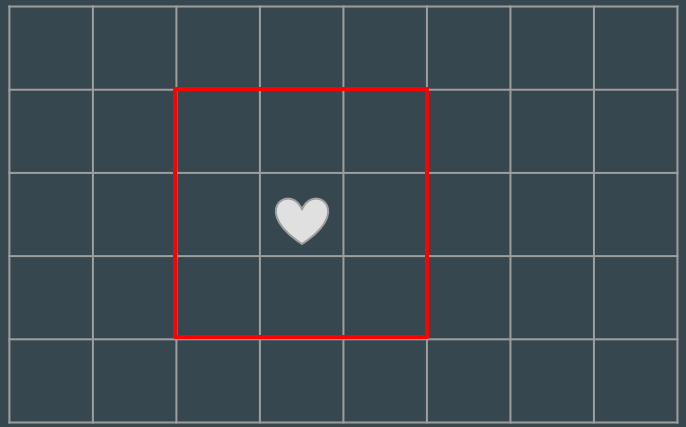
The tilemap defines the world, while the camera determines which part of that world the player sees. Scrolling creates the illusion of exploring a much larger scene without displaying the entire scene at once.

Tilemap Scrolling

Mathematically, if a tile coordinates on the real world is (x_w,y_w) and the camera is at coordinates (x_c,y_c) then its screen position is:

$$x_s = x_w - x_c \quad , \quad y_s = y_w - y_c$$

Example: Tile size = 32x32 , real world coordinates of the heart is (96, 64), the camera coordinates (64, 32) so the heart is on the screen
at coordinate: $x_s = 96 - 64 = 32$ $y_s = 64 - 32 = 32$
Screen coordinates of the heart (32, 32)



Visualizing a Scene

Visualizing a Scene

In a video game, a scene can display tiles and sprites simultaneously.

A tile-based scene generally contains the following elements:

1. Tile Dimensions
2. The Tileset
3. Scene Dimensions: Map Size
4. The Visual Grid: The visual grid specifies which tile must be displayed at each position in the map.
5. The Logical Grid: The logical grid describes the properties of each cell according to the game rules, independently of its visual appearance.

Visualizing a Scene

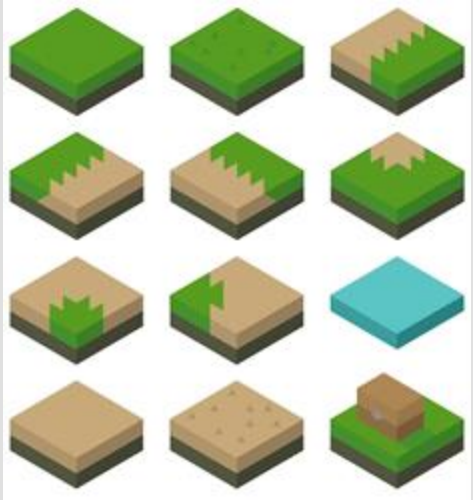
Why separate the visual and logical grids?

Because a tile's appearance does not necessarily determine its behavior.

For example, a tree may be represented visually by a tile and logically by an obstacle that the player cannot cross.

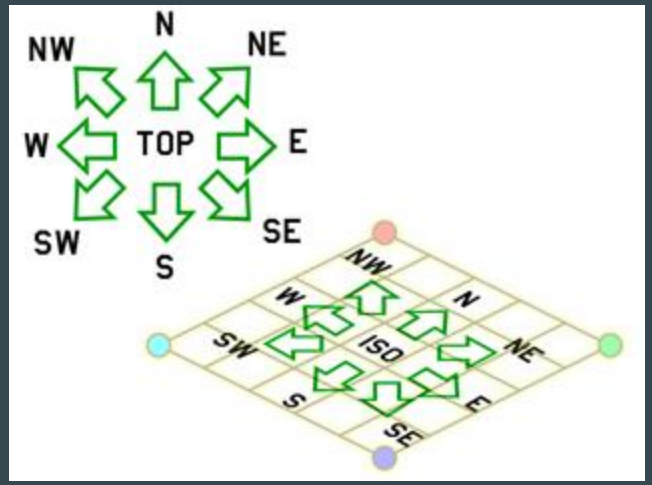
Isometric Tiles

Isometric tiles are designed to create the illusion of a 3D world in a 2D game. Instead of looking at the world from directly above or from the side, we see it from an elevated angle, so we can see the tops and sides of objects.



Isometric Tiles

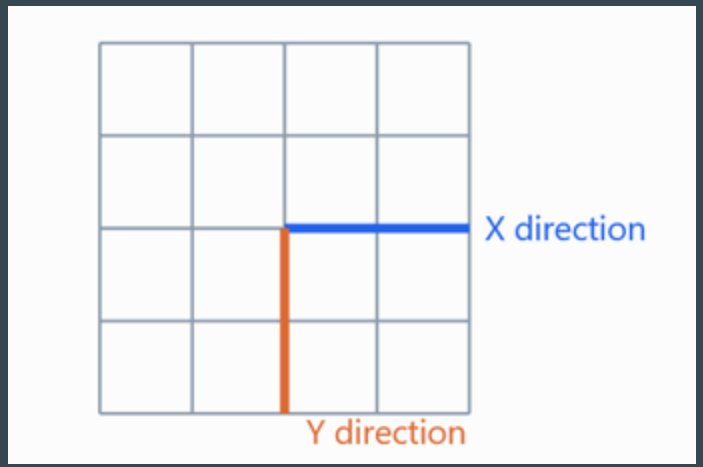
In a normal grid, each tile is a square. In an isometric grid, a ground tile is commonly represented by a diamond shape. The world appears to have depth and height.

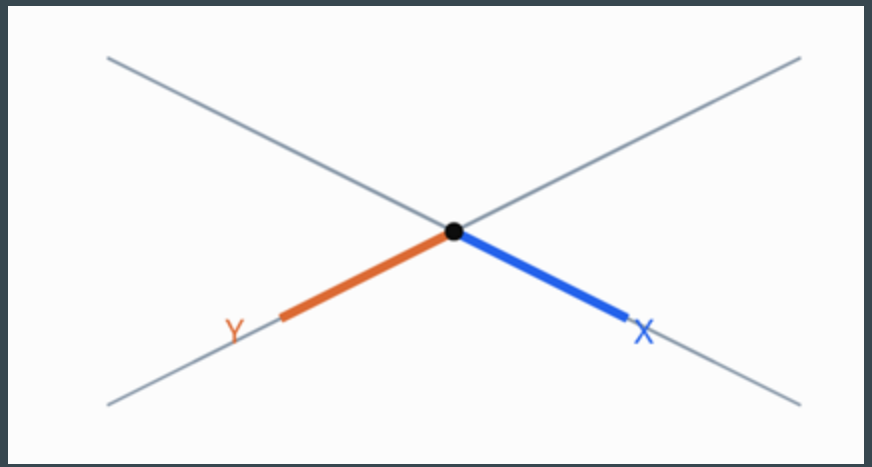




Isometric Tiles: How do we calculate the position of a tile ?

Orthogonal tiles use a regular square grid on screen, while isometric tiles use a **diagonal projection** to create the illusion of depth. Both can be used to build large tilemaps and support scrolling.





Isometric Tiles: How do we calculate the position of a tile ?

Suppose each diamond tile is *w* pixels wide and *h* pixels high.

Let *(i,j)* be the tile's grid coordinates. Its screen position can be calculated using:

$x = (i - j)\ w / 2 \qquad y = (i + j)\ h / 2$

The grid is mapped onto the screen so that the two axes appear diagonal and the square cells look like diamonds.

Example of a 2D Game: Practice